

Hamid Achahboune

RAPPORT DE PROJET TRADECORP

Projet : Pipeline ETL Enterprise TradeCorp

Technologies : PySpark, Apache Airflow, Azure ADLS Gen2, Docker, PyTest

Architecture : Lakehouse Medallion (Bronze / Silver)

Formation Data Engineer Simplon

Sommaire et Matrice des Compétences

Section du Rapport	Compétences associées	Éléments clés apportés par le projet
1 & 2. Contexte & Architecture	C24, C25	Choix du modèle Lakehouse Medallion (Bronze/Silver).
3, 4 & 6. Ingestion, Azure ADLS Gen2 & Fallback	C27	Synchronisation ADLS Gen2 et bascule automatique local/cloud.
5 & 7. Docker & Gestion I/O Temporaire	C31	Purge <code>rmmtree()</code> , optimisation des E/S et démarche Green IT.
8 & 9. Modélisation & Grande Jointure Snowflake	C26	Modèle Snowflake (<code>build_enriched</code>), typage <code>to_date</code> , déduplication.
10. Enrichissement Monétaire & Multi-Devises	C26, C28	Dépilage dynamique JSON/STRUCT/MAP (<code>hasattr</code>) et calcul local.
11. Écriture Parquet	C28	Persistance orientée colonne (Zone Silver).
12. Orchestration Airflow	C26, C28, C31	Exécution conteneurisée via <code>DockerOperator</code> (<code>auto_remove=True</code>).
13. Assurance Qualité & CI/CD	C32	Tests unitaires isolés PyTest avec fixture Spark <i>in-memory</i> .

1. Présentation Générale du Projet (Compétences C24, C25)

Le projet TradeCorp s'inscrit dans le cadre d'une modernisation de la plateforme décisionnelle et analytique d'une entreprise de commerce international. L'objectif principal est de concevoir et déployer un pipeline de traitement de données massif (ETL) capable d'ingérer, nettoyer, enrichir et consolider des volumes importants d'ordres de vente, de catalogues produits et de taux de change internationaux.

Ce système repose sur une architecture moderne de type **Lakehouse Medallion**, s'appuyant sur l'écosystème **Apache Spark (PySpark)** pour la puissance de calcul distribué et **Apache Airflow** pour l'orchestration robuste des workflows complexes.

2. Architecture Globale de la Solution (Compétences C24, C25)

L'architecture adopte un modèle découplé et scalable garantissant la reproductibilité, l'isolation des dépendances et la sécurisation des échanges :

- **Source de Données** : Fichiers CSV hétérogènes (commandes, clients, employés, produits) et fichiers JSON issus d'API financières (taux de change).
- **Moteur de Traitement (PySpark)** : Scripts modulaires de transformation exécutés au sein de conteneurs Docker dédiés.
- **Stockage Cloud (Azure ADLS Gen2)** : Organisation en containers Medallion : **raw** (Données brutes / Bronze) et **clean** (Données transformées / Silver).
- **Orchestrateur (Apache Airflow)** : Supervision séquentielle via des opérateurs Docker conteneurisés.

3. Ingestion des Données Multi-Sources (Compétence C27)

Le pipeline traite deux grandes familles de sources :

- **Données Relationnelles Métier (CSV)** : Tables brutes contenant l'historique commercial (**orders**, **order_details**, **customers**, **products**, **categories**, **employees**, **shippers**, **country_currency**).
- **Données Financières Externes (JSON)** : Flux de taux de change récents extraits d'API et normalisés dans le fichier **exchange_rates.json**.

4. Stockage Cloud Azure ADLS Gen2 & Organisation Medallion (Compétence C27)

Le système s'appuie sur Azure Blob Storage / Data Lake Storage Gen2 pour assurer la persistance haute disponibilité des jeux de données :

- **Zone Bronze (Container **raw**)** : Ingestion brute des fichiers source sans altération du schéma d'origine.
- **Zone Silver (Container **clean**)** : Stockage des données nettoyées, consolidées et enrichies au format optimisé Parquet (**tradecorp_enriched.parquet**).

5. Conteneurisation & Environnement Docker (Compétence C31)

L'ensemble du stack technique est conteneurisé afin d'éliminer le problème du *"works on my machine"*. L'image de référence `projet-tradecorp-spark` embarque l'environnement Python 3.x, Apache Spark 3.x et les connecteurs requis (Azure Storage SDK).

6. Stratégie de Gestion des Erreurs et Fallbacks Azure / Local (Compétence C27)

Afin de garantir une résilience maximale de l'ETL en environnement de production et d'intégrer des scénarios dégradés (panne réseau Azure, indisponibilité temporaire des credentials) :

- Lors de la lecture du fichier `exchange_rates.json`, si la connexion au container Azure `raw` échoue, le système bascule de manière transparente sur une version de secours stockée en local sous `/home/jovyan/data/exchange_rates.json`.

7. Gestion du Stockage Temporaire et I/O Docker (Compétence C31)

Afin d'éviter tout conflit de permissions I/O avec le système de fichiers de l'hôte Windows/Linux et de limiter l'empreinte disque (**Green IT**) :

- Les écritures intermédiaires s'effectuent dans des répertoires temporaires dédiés au sein du conteneur (`/tmp/data/raw` et `/tmp/data/clean_output`).
- Une fonction de nettoyage systématique (`shutil.rmtree()`) est exécutée avant chaque cycle d'écriture pour purger les artefacts résiduels.
- Lors du téléversement final vers le container Azure `clean`, le système filtre automatiquement les fichiers techniques résiduels générés par Spark (tels que `.crc` et les répertoires `_SUCCESS` / checkpoints).

8. Modélisation des Données & Règles Métier (Compétence C26)

8.1 Nettoyage des Clients (`customers`)

- Déduplication stricte basée sur l'identifiant `customer_id`.
- Formatage syntaxique des noms de contacts en casse propre (`initcap`).
- Passage des codes/noms de pays en majuscules (`upper`) pour uniformiser les jointures ultérieures.

8.2 Consolidation des Commandes (`orders`)

- Conversion des colonnes textuelles de dates vers le type formel `DateTime` via `F.to_date()`.
- Création d'une colonne booléenne métier `is_shipped` égale à `True` si `shipped_date` n'est pas nulle, `False` sinon.

8.3 Calculs Financiers sur les Ventes (**order_details**)

Le calcul du sous-total net par ligne d'ordre intègre la remise appliquée (**discount**) selon la formule mathématique suivante :

`sous_total = round(prix_unitaire*quantite*(1-discount),2)`

9. Grande Jointure & Construction de la Table de Faits (Compétence C26)

Le cœur du traitement d'enrichissement réside dans la modélisation en étoile/flocon (**Snowflake Schema**). La table de faits principale est construite à partir de la table **order_details** associée aux ordres de vente via un **inner join**, puis enrichie par les tables de dimensions (**customers**, **products + categories**, **employees**, **shippers**) via des jointures à gauche (**left join**).

Table Dimension	Type de Jointure	Renommage Préventif (Résolution d'Ambiguïtés)
customers	LEFT JOIN	<code>company_name</code> → <code>customer_name</code> , <code>country</code> → <code>customer_country</code>
shippers	LEFT JOIN	<code>company_name</code> → <code>shipper_name</code>
employees	LEFT JOIN	<code>country</code> → <code>employee_country</code>
products & categories	LEFT JOIN	Jointure intermédiaire sur <code>category_id</code>

10. Enrichissement Monétaire & Taux de Change (Compétences C26, C28)

Le module **enrichement.py** gère l'enrichissement multi-devises et la conversion financière.

10.1 Sélection Dynamique de la Colonne Pays

Le script identifie automatiquement la colonne de destination prioritaire (**ship_country** s'il existe, sinon **customer_country**).

10.2 Fallback de Devise par Défaut

Si le pays associé ne possède pas de correspondance dans la table `country_currency`, la devise affectée par défaut est l'USD via la fonction `F.coalesce(F.col("currency"), F.lit("USD"))`.

10.3 Ingestion Dynamique de Structures Complexes (JSON / STRUCT / MAP)

Les taux de change peuvent se présenter sous diverses formes selon la source. Le module analyse dynamiquement le schéma PySpark (`hasattr(rates_schema, "fields")`):

- **Cas STRUCT (JSON API imbriqué)** : Reconstruction explicite sous forme de MAP clé-valeur via `F.create_map()` puis dépilage par `F.explode()`.
- **Cas MAP** : Dépilage direct de la carte de taux.
- **Cas Plat (Mock/Test)** : Sélection directe de la colonne `rate` ou `exchange_rate`.

10.4 Conversion & Calcul du Sous-Total Local

Le montant converti en monnaie locale (`sous_total_local`) est calculé avec repli automatique sur un taux égal à 1.0 en cas d'absence de valeur :

```
sous_total_local=round(sous_total*coalesce(rate,1.0),2)
```

11. Écriture Optimisée au Format Parquet (Compétence C28)

Le jeu de données enrichi est sauvegardé sous forme de fichiers Parquet compressés (Snappy). Ce format en colonnes garantit des performances de lecture et des taux de compression optimaux pour les outils analytiques modernes (Power BI, Databricks, Snowflake).

12. Orchestration du Pipeline sous Apache Airflow (Compétences C26, C28, C31)

Le DAG `tradecorp_etl_pipeline` gère l'exécution planifiée quotidienne (`0 6 * * *`) du workflow ETL.

12.1 Stratégie DockerOperator

Chaque tâche Airflow s'exécute au sein d'un conteneur Docker éphémère basé sur l'image `projet-tradecorp-spark` via la socket Docker de l'hôte (`unix:///var/run/docker.sock`).

- **Isolation Totale** : Aucune pollution de l'environnement Airflow par les dépendances PySpark.
- **Nettoyage Automatique** : Le paramètre `auto_remove=True` garantit la destruction du conteneur après exécution.
- **Transmission Sécurisée** : Injection dynamique des variables d'environnement (clés Azure ADLS) extraites du fichier `.env`.

12.2 Enchaînement Sécurisé du DAG

Bash

```
fetch_exchange_rates_task >> reader_task >> transformer_task >> writer_task
```

13. Assurance Qualité & Tests Unitaires PySpark (Compétence C32)

La qualité du code et la non-régression sont assurées par une suite de tests unitaires sous `pytest` (ex: `test_transformers.py`).

- **Fixture Spark Session** : Instanciation d'une session PySpark locale unique partagée à portée de session (`@pytest.fixture(scope="session")`).
- **Execution In-Memory** : Les tests s'exécutent en mémoire sans dépendance vis-à-vis d'Azure ou du réseau, garantissant une exécution ultra-rapide en CI/CD.

14. Perspectives & Améliorations Futures

- **Gestion du Delta Lake** : Évolution du format Parquet vers Delta Lake pour introduire la gestion des transactions ACID et le *Time Travel*.
- **Alerting Slack / Teams** : Ajout de callbacks sur échec (`on_failure_callback`) dans Airflow pour notifier l'équipe Support en temps réel.
- **CI/CD Automatisé** : Intégration de GitHub Actions pour l'exécution automatique des tests unitaires PyTest à chaque Pull Request.